

What we built, how to show it to sir, and what the frontend should do

ProvGuard is a Java agent. It watches three risky Java methods while a program is running, writes a safe log, and can stop only a high-confidence rule match. The frontend does not exist yet. It should display that log. It should not reimplement the agent.

1. The problem we solved

A Java program can start an operating-system process or load a class during deserialization. Those calls are a common step in command injection and unsafe deserialization. ProvGuard answers one practical question: when one of those methods is called, which method path reached it, how risky does that path look, and should the call be allowed to continue?

The agent records a call-path trace with `StackWalker`. That is the stack of Java methods that led to the call. It is not a provenance graph, and it is not taint tracking. The argument is not stored in full. The log keeps a SHA-256 hash, the length, and a short redacted preview.

Scoring has two parts. Deterministic rules look for command keywords, reflection, repeated calls, and a deserialization denylist. A Java Isolation Forest adds an anomaly score from call-path features. Monitor and Alert modes never stop the program. Block mode throws `ProvGuardBlockedException` only when the rule score is at least 80, or when the rule score is 60–79 and the model score is also at least 80. A high model score by itself never blocks. If the agent, the log, or the model fails, the target program continues.

2. Exactly what is already done

The implementation is Java 21, built with Maven, and verified with `mvn verify` on JDK 26. The requirement document mentioned Python for training. This version trains and scores the Isolation Forest in Java, because the team asked for a Java-only project.

Piece	Where it lives	What it does
Agent	provguard-agent	premain, Byte Buddy hooks, fast path on the application thread
Three sinks	advice classes	All Runtime.exec overloads, ProcessBuilder.start, ObjectInputStream.resolveClass
Rules and modes	provguard-rules	Monitor, Alert, Block. Fail-open. Recursion guard.
Log	provguard-storage	Bounded queue, one background worker, events.jsonl
Model	provguard-ml	Java Isolation Forest packaged inside the agent
Demos	provguard-demo-app	Safe echo, suspicious echo, denylist deserialization class
Tests	unit tests and provguard-it	Second JVM started with -javaagent
Docs	docs/ and README.md	Architecture, class, and sequence diagrams; test map; viva notes

Measured on this machine: startup 147.5 ms without the agent and 2059.3 ms with it, because the agent installs its hooks. An allowed echo call went from 87.3 ms to 98.8 ms, about 13%. On the held-out dataset, mean suspicious score 94.0, mean held-out benign score 51.7, recall at 80 is 1.00, false-positive rate at 80 is 0.13. Say these numbers as measurements. Do not say the overhead is low, and do not say the model catches every attack.

Not in this version, and do not claim them: JNDI coverage, a dashboard, ONNX, dynamic attach, taint tracking, a provenance graph, Log4Shell coverage, exploit payloads, a slide deck, and a demo video. The dashboard is the frontend teammate's job.

3. How to show this to sir

Give him the project folder, or a GitHub link after the team pushes it. The code is complete locally and has not been pushed yet. Open the README, then run the three demos live. After each run, open events.jsonl in the project root.

```
mvn verify

.\scripts\run-demo.ps1 -Mode MONITOR -Demo safe
.\scripts\run-demo.ps1 -Mode ALERT -Demo suspicious
.\scripts\run-demo.ps1 -Mode BLOCK -Demo blocked
```

Demo	What sir should see
Safe, Monitor	The program finishes. The log has <code>PROCESS_BUILDER_START</code> , <code>RUNTIME_EXEC</code> , and <code>DESERIALIZATION_RESOLVE_CLASS</code> . The raw command text is not stored. <code>modelStatus</code> is <code>AVAILABLE</code> .
Suspicious, Alert	The echo still runs. The log is an alert because the argument text contains command keywords. Nothing is blocked.
Denylist, Block	Deserialization of <code>BlockedDemoObject</code> is denied. The log has enforced: <code>true</code> and <code>UNSAFE_DESERIALIZATION_CLASS</code> . The object has no dangerous <code>readObject</code> .

Two-minute explanation, also written in `docs/viva-notes.md`: ProvGuard instruments process execution and deserialization class resolution. On each call it captures the stack, stores a hash and a redacted preview, scores known patterns with rules, and adds an Isolation Forest score. Only a confirmed rule match can stop the call, and only in Block mode.

Files to leave open: `README.md`, `docs/architecture.md`, `docs/class-diagram.md`, `docs/sequence-diagram.md`, `docs/limitations.md`, `reports/ml-evaluation.md`, `reports/performance-results.md`.

If the college also wants slides, use this order: problem, three sinks, call-path trace, safe metadata, rules and the block policy, Java Isolation Forest, three live demos, measured numbers, honest limits. A video can be a screen recording of the three commands above. Those two files are the only submission pieces still missing.

4. What to ask your teammate to do now

Send her this PDF and the repository. Ask her to build a read-only web frontend that loads `events.jsonl` and explains the three demos. She should not change the agent, the rules, or the model. She should not add a way to run commands from the browser.

Say this to her:

“The Java agent is finished. Please build the presentation frontend. It reads the JSONL file the agent already writes. Each line is one monitored call. Show the call path, the rule score, the model score, the decision, and whether the block was actually enforced. Include a page that walks a viewer through the safe demo, the alert demo, and the block demo. Do not store or display a full command. The log only has a hash, a length, and a 20-character redacted preview. Do not call the stack a provenance graph. Do not let the model score look like it can block by itself.”

Screens to build

1. **Home.** One sentence on the problem, the three monitored methods, and the three modes. A button to load `events.jsonl` from disk.
2. **Run summary.** Counts of allowed, alerted, and blocked events. Show `modelStatus`. If it is `ML_UNAVAILABLE`, say the rules decided.
3. **Event table.** Time, sink, path signature, rule score, ML score, decision, enforced. Filters for sink and decision.
4. **Event detail.** The call frames from outermost application method to the sink. Matched rule names. The explanation string. Hash, length, and redacted preview. Requested class name for deserialization.

5. **Demo guide.** Three cards: Monitor plus safe echo, Alert plus suspicious echo, Block plus BlockedDemoObject. Each card says what the viewer should see in the log.
6. **Limits.** Short text from docs/limitations.md so the UI does not overclaim.

Data she should use

The agent appends one JSON object per line to events.jsonl. Sample runs are already in reports/output/. She can develop against those files before she runs Java. Important fields:

Field	Use it for
sinkType	RUNTIME_EXEC, PROCESS_BUILDER_START, or DESERIALIZATION_RESOLVE_CLASS
pathSignature	The call path, for example SafeDemoApplication>Runtime.exec
callFrames	Class, method, and line for the detail view
argumentHash, argumentLength, redactedArgumentPreview	The only argument display. Never invent the original command.
requestedClassName	The class seen during deserialization
ruleScore, mlScore, finalScore	Scores from 0 to 100. mlScore may be null.
decision, enforced, mode	ALLOW, ALERT, or BLOCK. enforced: true means the call was actually stopped.
matchedRules, explanation, modelStatus	Why the event was scored. Status is AVAILABLE or ML_UNAVAILABLE.

She can use plain HTML, React, or any frontend she already knows. A local page that lets the user pick a .jsonl file is enough. A small local server is optional. The page must work when she opens a saved log, with no dependency on the Java process staying alive.

What she should leave alone

- Do not rewrite detection in JavaScript. The scores in the file are the result.
- Do not add a control that executes Runtime.exec or deserializes a class.
- Do not show a block banner when enforced is false. Alert mode can contain a block-shaped decision that was not enforced.
- Do not add exploit examples, shell payloads, or a “try an attack” box.
- One Runtime.exec on this JDK can also produce a PROCESS_BUILDER_START line, because the JDK implements exec with ProcessBuilder. The UI should show both lines. It should not hide one of them.

5. Suggested split for the viva

Person	Talks about
Agent owner	Instrumentation, rules, block policy, fail-open, the three live commands
Frontend teammate	How a reviewer reads an event: path, scores, decision, enforced flag, and why the raw command is absent

Until her screens exist, the submission is still the Java project plus this brief. The frontend is the presentation layer on top of a finished agent.